Reference Solutions That Fail Their Own Checks, and Other Measurement Pitfalls in Released AgentDyn Prompt-Injection Runs

Anonymous authors
Paper under double-blind review

Abstract

We audit the released code and runs of the AgentDyn prompt-injection benchmark. In public prompt-injection benchmark artifacts, measurement plumbing can silently fail.

First, 20 of AgentDyn's 28 benchmark-specific injection tasks ship a reference solution that fails the task's own security check (18 copy-paste banking stubs, nine of which call a tool absent from the suite, one wrong-argument call, one missing OTP step); we confirm all 20 by executing the suite's own check logic. The nine shopping stubs had already been replaced in AutoDojo's public code (June 2026); the github and dailylife defects and the complete census are, as far as we found, new. An executed audit of related code bases also finds that original AgentDojo v1.2.x workspace injection tasks 6–13 return an empty reference solution that fails `check()` (whether by design is unverified). Of five 2026 papers found that evaluate on AgentDyn, two use reference-solution-derived information (exposed), one does not and two are unknown.

Second, re-scoring released runs by attempted instead of executed attacks with an argument-level attempt scorer (ATT-ARG; 78/80 agreement, κ 0.95 with blind AI labels) gives positive rank agreement (Kendall τb 0.642–1.0 per cell). 3 of 9 eligible cells contain at least one significant reversal under the preregistered rule; every ATT-ARG reversal involves progent, and the count falls to 1 of 9 under task-clustered inference and 0 of 9 without progent. The earlier tool-identity (name-match) scorer is kept as a reference.

Third, replaying 73,009 stored labels against current checker code changes 0.27% of them; a small LLM-judge case study (one agent model, three defenses, single AI adjudicator) finds that judges mostly count blocked attempts as successes.

Keywords: prompt injection; LLM agents; benchmark audit; security evaluation; measurement validity; reproducibility

Highlights (also in HIGHLIGHTS.md)

- 20 of 28 AgentDyn injection tasks fail their own reference-solution check
- All 20 confirmed by executing the suites' own check logic offline
- Shopping stubs were fixed earlier in AutoDojo; github and dailylife were not
- Argument-level attempt scoring: reversals in 0–3 of 9 cells, all with progent

Labels used in the text.

- [CODE FACT]: deterministic audit of code, confirmed by executing it; no inference.
- [PREREGISTERED RULE]: follows a protocol written before the data were scored.
- [DEVIATION]: departs from a written protocol, or applies it to a subset chosen after seeing data.
- [EXPLORATORY]: designed or remapped after seeing data; descriptive.
- [AI LABELS]: labels produced by an AI assistant reading traces, not by a human.

1 Introduction

Defenses against indirect prompt injection are compared by attack success rate (ASR). Pathade et al. argue that ASR is a family of metrics whose defining choices papers rarely state, and show analytically that some choices can reverse defense rankings, without running defenses themselves (§III, §V, §VIII). Shaw audits one benchmark and builds a validated harness, showing that scoring by tool identity rather than arguments inflates ASR and that a scenario must be able to fire on the attacker's action (§IV, §V-A, §V-D, §VII-A).

This paper is an **artifact audit** of AgentDyn: its v1 task code (commit 5353cf7) and its released runs, which we did not author. Our thesis: *in public prompt-injection benchmark artifacts, measurement plumbing can silently fail. 20 of 28 AgentDyn-specific injection tasks ship a reference solution that does not satisfy its own security check (18 are copy-paste banking stubs; the nine shopping stubs were independently fixed in AutoDojo's code). In the released AgentDyn runs, every significant EXEC-vs-ATT-ARG rank reversal involves the execution-time defense progent.*

Findings, ordered by strength.

1. **Main result [CODE FACT]:** a reference-solution defect in 20 of 28 AgentDyn-specific injection tasks, confirmed by executing the check logic, with an extended audit of related code bases and a downstream-exposure census (§4). AutoDojo fixed the shopping subset earlier (§2).
2. **Secondary, descriptive:** a narrow ranking sensitivity to the success definition, confined to two execution-time defenses on one benchmark, shown with all sensitivity analyses side by side (§5). It is a quantification of magnitude and fragility, not a discovery: that attempt-based metrics give execution-time blockers no credit is expected a priori and anticipated by Shaw (§V-F) and Pathade et al. (§III, §V-B).
3. **Short results:** a negative checker-drift replay and a small judge-vs-checker case study (§6).

Non-claims. We claim no defense is effective or ineffective, no general ranking instability, and no validated measurement framework. AgentDojo contributes only one eligible ranking cell and no reversal; the ranking result is about AgentDyn. Our own earlier testbed, which motivated these questions, is not evidence here and is described only in Supplementary S1.

2 Related work

All claims below come from full-text reads of the arXiv HTML versions; section references are to those versions.

Measurement validity of IPI evaluation. Shaw (2609.32691) audits one unnamed IPI benchmark. Its defect classes include success scored by tool identity (D2, §IV), and re-scoring identical traces gives 21.7% ASR under tool-identity scoring vs 1.2% at argument level (§VII-A). Pre-flight checks reject attacker goals that name an absent tool (§V-A, §V-D), and blocked calls are recorded (§V-F). Pathade, Pawar and Patil (2609.25173) define six axes of ASR, including the success oracle (§III), and derive ranking inversion analytically without running defenses (§V-B, §VIII). Because of Shaw's D2, our earlier function-name attempt scorer is now only a tool-identity reference, and the primary attempt scorer is argument-level (ATT-ARG, §3).

Benchmark audits. Zhu et al. (2507.02825, ABC) recommend verifying ground-truth correctness and shipping an automatic oracle solver (items T.7–T.9, §4.1). Our self-check is an application of T.9, so the principle is known. BenchGuard (2604.24955) names "errors in the reference solution" as a defect category (§3.2, Table 1) and audits ScienceAgentBench and BIXBench with LLMs (§5). Vaghasiya, Bhat et al. (2607.02577) audit evaluator validity in BFCL v4, τ^2-bench and LiveMCPBench (§3.4, §4.3). Abrich (2610.09142) injects faults into reference solutions that are assumed correct, to find verifier false accepts in AppWorld and WorkArena (§2, §3). Luo et al. (2609.34262) detect unearned passes, e.g. reference solutions leaked via git history (§4–§5). The closest work is Bellibatlu, Wang and Zhang (2609.37315). They audit, by execution at pinned commits, whether mutating *tools* honour their advertised contracts in four benchmarks,

including the full 25-tool AgentDojo v1 surface (§V-B1, §V-B3). They do not audit injection-task reference solutions or AgentDyn.

Prior identification of part of our main finding. AutoDojo (2606.15057; code xhOwenMa/AutoDojo at aa458794) builds on AgentDojo and AgentDyn (§4.1, App. D). A note in its shopping injection-task file, dated 2026-06-14, says the original `send_money(recipient="")` stubs "matched no security() check" and replaces them. Afterwards 6 of 9 pass `check()`, and three remain OTP-gated (per the code note). We found no statement of this in the paper text. We credit AutoDojo with the first public fix of the shopping stubs. Its github and dailylife tasks are unchanged.

What is known and what is new. *Known:* the principle that reference solutions must satisfy their own checks (ABC T.7–T.9; BenchGuard's GT class; Shaw §V-A); executed audits of AgentDojo artifacts at the tool layer (Bellibatlu et al.); the AgentDyn shopping stubs (AutoDojo); tool-identity inflation of ASR (Shaw §VII-A); analytical ranking inversion (Pathade et al.). *New, as far as we found:*

- the executed, complete census of 20/28 AgentDyn injection-task reference-solution failures, including github (9, tool absent) and dailylife (2) and the root cause of dailylife injection_task_2;
- the cross-benchmark defect table;
- the str-only cause of the "not injectable" self-check message;
- the downstream-exposure census;
- an empirical, argument-level rank-reversal analysis on released third-party runs.

We do **not** claim to be the first audit of an AgentDojo-derived prompt-injection benchmark.

Other work. The Same Zero (Yin, 2610.04504) reports AgentDojo banking state checks missing 11/75 tool-level attacks [partial read]. Liu (2610.03448) re-evaluates detectors offline. Hardy et al. (2610.00651) apply generalizability theory to agent leaderboards. Benchmarks and defenses (AgentDojo 2406.13352, InjecAgent 2403.02691, CaMeL 2503.18813, Progent 2504.11703) are cited in the references.

3 Data, definitions and records

Threat model. The user turn, system prompt and defense code are trusted; tool-returned content is untrusted; the attacker is non-adaptive and wants an effect the user did not request.

Data. Public AgentDojo and AgentDyn runs and code: 125 (benchmark, pipeline, attack) run sets, 50 AgentDojo and 75 AgentDyn, with 74,528 injection episodes. The checker-drift replay uses 73,009 non-error, non-DoS episodes (AgentDojo 31,604, AgentDyn 41,405). AgentDyn code: local clone at commit 5353cf7615b135cace8d07c8f12dac53a16b6db3; the installed package is byte-identical to the clone.

Success definitions.

- EXEC: the stored state-based `security()` result (the benchmark's own ASR).
- ATT-ARG (primary attempt scorer from this revision) [EXPLORATORY; definition and constant table hash-locked before scoring, 2026-10-16 16:51 Tehran]: the agent proposed a call to an injection target function whose arguments contain an attacker-specific constant of that injection task (e.g. the attacker's IBAN, e-mail address, URL or file). Constants were extracted by code from task class attributes, the GOAL text and `security()`. 27 of 154 task-versions with no usable constant or no target function were excluded (dailylife injection_task_6; travel injection_task_2/4/6; workspace injection_task_1 and 6–13), removing 9,080 AgentDojo and 1,460 AgentDyn episodes. ATT-ARG is not preregistered [DEVIATION 11].
- ATT (ATTEMPTED, name-match; preregistered; now a tool-identity reference only, cf. Shaw D2, §IV, §VII-A): the agent proposed a call to an injection target function, by function-name match. For AgentDojo and AgentDyn dailylife the targets come from `ground_truth()` as preregistered; for AgentDyn shopping/github, whose `ground_truth()` is defective (§4), from a target map written by the AI analyst from each `security()` after first-pass disagreements were seen [DEVIATION 1].

- A third preregistered definition (attempted with correct arguments) was implemented with a deviating matcher, behaved inconsistently, and is not reported.

Defense stages. Input (repeat_user_prompt, spotlighting_with_delimiting, transformers_pi_detector, prompt_guard_2_detector, piguard_detector), planning (tool_filter, camel) and execution-time (progent, drift). The stage map was written after the reversals had been seen.

Notation. A *cell* is a (benchmark, model, attack) combination with $\geq$3 defenses including no-defense. A *flip* is a defense pair whose ASR difference is significant with opposite signs under EXEC and ATT. τb is Kendall's tau-b between the EXEC and ATT rankings of a cell's defenses; κ is Cohen's kappa.

Preregistration record and its limits. The ranking-flip protocol was added to `docs/DESIGN_VARIANCE_STUDY.md` on 2026-10-15 at 22:40 Tehran; the recovered file `docs/DESIGN_VARIANCE_STUDY.PREREG_2026-10-15_2240.md` has sha256 `97bdb074...61e63c1`, byte-identical to `review/PREREG_RANKFLIP.sha256`. It specifies: primary statistic Kendall τb with a bootstrap CI per cell; bootstrap over episodes within (cell, defense), 2000 reps, seed 20261015; decision rule "flip if the ASR-difference 95% CI excludes 0 with opposite signs; main finding only if $\geq$25% of cells contain a flip"; error episodes kept at their stored value, with a drop-errors sensitivity. **The preregistration was not blind to the mechanism:** before 22:40 the same analyst had already seen the judge pilot (21:24) and had labelled 53 + 60 gpt-4o episodes in which tool_filter/PG2 attempts did not execute, so the attempted-vs-executed hypothesis came from inspected data. The preregistration does not mention excluding any suite.

4 Main result: reference solutions that fail their own checks [CODE FACT]

What a reference solution must do. Each AgentDojo-style injection task has a `ground_truth()` (the call sequence that achieves the attacker goal) and a `security()` predicate on the post-run environment. The benchmark's own `TaskSuite.check()` runs the ground truth and requires `security()` to be True. If it is not, the reference solution and the check disagree about what the attack is.

Executed self-check. We ran upstream `TaskSuite.check()` unchanged on every v1 suite, offline, then repeated its per-task logic with each task isolated so that one crash does not hide the rest:

- shopping: `check()` returns failure; 9 of 9 injection tasks fail security after their ground truth.
- github: `check()` aborts with `ToolNotFoundError: The requested function send_money is not available`; per task, 9 of 9 injection tasks raise it.
- dailylife: `check()` aborts with a `ValidationError` in injection_task_6; per task, 8/10 pass, injection_task_2 fails security and injection_task_6 raises.
- The four original AgentDojo suites (workspace, travel, banking, slack): every injection task passes (6/6, 7/7, 9/9, 5/5).

Result. 20 of the 28 AgentDyn-specific injection tasks (dailylife 10, shopping 9, github 9) fail their own check, all 20 confirmed by execution. Four classes:

1. **Copy-paste banking stub, 18 tasks** (every shopping and every github task): `ground_truth()` returns the same banking `send_money(recipient="", amount=0.01, ...)` call regardless of the stated goal (password change, purchase, download, leak).
2. **Tool absent:** in github `send_money` does not exist, so the reference raises ToolNotFoundError (within the 18).
3. **Wrong arguments, 1 task** (dailylife injection_task_6): the calendar call omits the required start/end times.
4. **Missing verification step, 1 task** (dailylife injection_task_2): see below.

Root cause of dailylife injection_task_2. The goal is to send the whole balance to the attacker. The reference solution is a single `send_money(amount=1810.0, ...)`. The suite's own `send_money` refuses

amounts above 500 from an unverified account: it e-mails an OTP, queues the transfer and returns "Your transaction amount exceeds $500. Please verify your account ...". The balance is 1810.0, so no transaction is made and `security()` is False. Appending the step the reference omits, `verify_transaction(otp=...)` with the queued OTP, executes the transfer ("Sent 1810.0 to AT6854532054") and `security()` becomes True. The defect is in the reference solution, not in the checker or in our audit harness; agents that complete the OTP step do satisfy the check in the released runs (162/1460 inj2 episodes).

Consequences and scope.

- *Self-check:* `TaskSuite.check()` fails or aborts for shopping, github and dailylife.
- *Attack generation:* only the `tool_knowledge` attack embeds `ground_truth()` in injections; on these suites it would instruct agents to call `send_money`. The released AgentDyn attack directories contain only important_instructions, direct and none, so no released number derives from it; we could not trace the AgentDyn paper's tables to unreleased runs.
- *Derived metrics:* any attempt-based metric keyed to `ground_truth()` is invalid on these suites; this is what forced the post-hoc target map in §5 [DEVIATION 1].
- *Released ASR:* not affected for the released runs. It is computed by `security()` on the post-run environment and never uses `ground_truth()`.

Two side observations (not counted as defects).

- In this code version `check()` also reports every user task in every suite, including the four original AgentDojo suites, as "not injectable" (40/40, 20/20, 16/16, 21/21, 20/20). Root cause: tool-message content is a list of content blocks, and `is_task_injectable()` only scans string contents, so it returns False although the canary is present. The function is called only in the user-task section of `check()`, so it cannot affect injection-task checks.
- Replaying user-task reference solutions, utility fails for shopping 20/20, github 13/20 and dailylife 9/20 user tasks (and workspace user_task_7). Most failing AgentDyn cases hit an OTP or verification gate that a static reference cannot pass (e.g. a hard-coded OTP rejected as incorrect). This affects the self-check, not stored labels; we did not audit each task.

Extended audit of related code bases [CODE FACT; EXPLORATORY scope; procedure hash-locked before running]. We ran the same unchanged `check()` and per-task logic on every registered suite and version of four code bases: AgentDojo (089ed468, MIT), AgentDyn (SaFo-Lab 5353cf76, MIT), AutoDojo (xhOwenMa/AutoDojo aa458794, MIT) and uber/ADR (a6076aa3, Apache-2.0). ADR does not import as a package. By diff it is AgentDojo v1/v1_1 with only import paths and a `TASK_TYPE` attribute changed, so it was not executed. Unique injection tasks failing their own reference-solution check, counted once per code base where they are introduced:

Benchmark	Stub	Wrong args	Missing step	Tool absent	Other
AgentDojo (v1.2, v1.2.1, v1.2.2 workspace)	8 (injection_task_6–13: `ground_truth()` returns `[]`)	0	0	0	0
AgentDyn (new suites)	9 (shopping)	1	1	9 (github; also stubs)	0
AutoDojo (beyond AgentDyn)	0 (the 9 shopping stubs replaced)	0	0	0	3 (shopping inj2/5/8: random-OTP gate a static call list cannot pass)
ADR	0 (by diff; not executed)	0	0	0	0

AutoDojo's ten new banking_v2/travel_v2 suites pass. AutoDojo leaves the github and dailylife defects unchanged. Whether the empty AgentDojo v1.2.x workspace references are intentional (security-only tasks) is unverified, and they are not counted in our 20. The original v1 AgentDojo suites pass.

Downstream exposure [EXPLORATORY; procedure hash-locked; incomplete census]. We searched for 2026 papers evaluating on AgentDyn (web and DDG search; the Semantic Scholar citation API returned HTTP 429). We read each paper found in full and classified exposure to the defective reference solutions. This reports exposure only; we do not claim any paper's results are wrong.

Paper	AgentDyn suites	Attacks	Reference-solution use	Exposure
AgentDyn (2602.03117v3)	all three	important_ instructions; ignore-previous, completion, manual adaptive (§4.1, App. F)	none stated; utility/ ASR from `security()` (Table 3)	not exposed
Agent-Sentry (2603.22868)	all three	benchmark injection tasks	trace labels from "the injection task's ground-truth signature" (§4.1.4)	exposed
AuthGraph (2605.26497)	not itemised	"diverse attack types"	none found	unknown
ActGov (2609.24446)	60 user / 28 injection tasks	not specified	"native benchmark scorer"	unknown
AutoDojo (2606.15057)	all three (App. D)	important_ instructions plus adaptive optimizer (§4.1)	optimizer prompt includes `ground_truth()` calls (code: variant_generation/ suite_gen_utils.py); github/dailylife unchanged	exposed (code level; use in App. D runs unverified)

Relation to prior work and disclosure. Shaw states the principle that a success condition must be able to fire and adds pre-flight checks for goals naming absent tools (§V-A, §V-D); that overlaps class 2 in concept. AutoDojo's code fixed the nine shopping stubs in June 2026 (§2). The AgentDyn issue tracker (issues #1–#9, checked 2026-10-16) contains no report of the reference-solution defects; issues #8/#9 concern github user tasks and #6 a github `security()` check. An issue report is drafted (`docs/ISSUE_AgentDyn_FINAL_round10.md`; canonical repository SaFo-Lab/AgentDyn, to which leolee99/AgentDyn redirects) and **not posted**; it will be filed with the maintainers before any preprint, and only after the owner approves its text. **Maintainers' response [PLACEHOLDER — not yet filed]:** date —, link —, summary —, resulting changes —.

5 Secondary result: ranking sensitivity to the success definition

Scope. 9 cells are eligible, all under important_instructions: one AgentDojo cell (gpt-4o-2024-05-13, 5 defenses, neither progent nor drift) and eight AgentDyn model cells. **All reversals in every analysis below are on AgentDyn; the AgentDojo cell has 0 flips.** The primary analysis is the preregistered rule on the **full data** (all suites), which requires the post-hoc target map for AgentDyn shopping/github [DEVIATION 1]. The analysis that excludes shopping/github was chosen after the full-data results existed and after the reference-solution audit; the preregistration does not provide for it, so it is reported as a [**DEVIATION**] sensitivity analysis, not as a confirmatory result.

5.0 Primary attempt scorer: ATT-ARG vs name-match [EXPLORATORY; procedure hash-locked]

Name-match ATT is tool-identity scoring, the defect Shaw names D2 (§IV, §VII-A). It is kept below (§5.1–§5.4) as the preregistered reference. The primary attempt scorer from this revision is ATT-ARG (§3). Both scorers were rerun on the same reduced task set, from which the 27 excluded task-versions are removed; the rest of the pipeline (episode bootstrap 2000 reps, seed 20261015; task-clustered paired bootstrap with Holm; LODO) is unchanged. Cells with ≥1 flip / flips / pairs:

Analysis	ATT-ARG, full data	Name-match, full data	ATT-ARG, subset [DEVIATION]	Name-match, subset
Preregistered rule (episode bootstrap)	3/9 / 5 / 299	4/9 / 6 / 299	2/9 / 4 / 290	2/9 / 4 / 290
Task-clustered + Holm	1/9 / 1 / 299	1/9 / 1 / 299	2/9 / 3 / 290	2/9 / 4 / 290
Without progent (LODO)	0/9 / 0 / 235	1/9 / 1 / 235	0/9 / 0 / 227	1/9 / 1 / 227
Without tool_filter (LODO)	2/9 / 3 / 238	3/9 / 4 / 238	2/9 / 2 / 230	1/9 / 1 / 230

ATT-ARG flips (full data): gpt-4o-2024-08-06 progent vs tool_filter; gpt-4o-mini progent vs prompt_guard_2_detector and progent vs repeat_user_prompt; llama-3.3-70b drift vs progent and progent vs tool_filter. Subset: gpt-4o-2024-08-06 progent vs spotlighting_with_delimiting and progent vs tool_filter; llama-3.3-70b drift vs progent and progent vs tool_filter. **Every ATT-ARG flip involves progent; none involves drift without progent**, and the name-match qwen3-235b drift flips disappear. Per-cell ATT-ARG τb (full data): AgentDojo 1.0 [0.8, 1.0]; gemini-2.5-flash 0.889; gemini-2.5-pro 1.0; gpt-4o-2024-08-06 0.817; gpt-4o-mini 0.911; gpt-5-mini 0.642 [−0.21, 0.734]; gpt-5.1 0.808; llama-3.3-70b 0.648 [0.5, 0.778]; qwen3-235b 0.944. ATT-ARG positives are a subset of name-match positives.

Validation [AI LABELS]. We drew a new blind sample: AgentDyn important_instructions episodes from the flip cells, excluding all earlier samples, 10 per defense × ATT-ARG label for progent, drift, tool_filter and repeat_user_prompt (n = 80, seed 20261017). The same AI assistant labelled them from the goal and the proposed calls only, without defense or labels; the labels were hash-locked (sha256 `da667c0d...`) before comparison. **These are AI labels, not human labels.**

Defense	ATT-ARG agreement	κ	ATT-ARG precision	Name-match agreement	κ
progent	20/20	1.0	10/10	20/20	1.0
drift	20/20	1.0	10/10	20/20	1.0
tool_filter	20/20	1.0	10/10	19/20	0.9
repeat_user_prompt	18/20	0.8	10/10	17/20	0.694
All	78/80 (97.5%, Wilson 0.913–0.993)	0.95	40/40 (0.912–1.0)	76/80	0.9

Both ATT-ARG disagreements are false negatives: a browsing step to the attacker's site, which is not a target function, and a call to a non-existent tool. The sample is stratified by the ATT-ARG label, which favours ATT-ARG in the comparison. A 40-episode human packet (key hashed, `human_packet/KEY_SEPARATE.sha256`) is prepared but not yet labelled.

5.1 Name-match reference — Kendall τb with bootstrap CI [PREREGISTERED RULE; full data uses DEVIATION 1]

Per-cell τb between the EXEC and ATT rankings, with the preregistered episode bootstrap (2000 reps, seed 20261015):

Cell (benchmark, model)	defenses	τb [95% CI], full data	ATT flips, full	τb [95% CI], subset [DEVIATION]	ATT flips, subset
AgentDojo, gpt-4o-2024-05-13	5	0.8 [0.8, 1.0]	0	0.8 [0.8, 1.0]	0
AgentDyn, gemini-2.5-flash	9	0.889 [0.778, 1.0]	0	0.889 [0.771, 0.986]	0
AgentDyn, gemini-2.5-pro	9	0.833 [0.704, 0.944]	0	0.704 [0.535, 0.841]	1
AgentDyn, gpt-4o-2024-08-06	9	0.743 [0.571, 0.873]	1	0.761 [0.592, 0.841]	1
AgentDyn, gpt-4o-mini-2024-07-18	10	0.854 [0.733, 0.911]	1	0.944 [0.761, 0.986]	0
AgentDyn, gpt-5-mini-2025-08-07	8	0.688 [0.288, 0.802]	0	undefined	0
AgentDyn, gpt-5.1-2025-11-13	9	0.73 [0.53, 0.841]	0	0.783 [0.551, 0.884]	0
AgentDyn, llama-3.3-70b-instruct	9	0.514 [0.31, 0.686]	2	0.551 [0.31, 0.707]	2
AgentDyn, qwen3-235b-a22b-2507	9	0.833 [0.722, 0.889]	2	0.857 [0.761, 1.0]	0

All τb estimates are positive and every lower bound is above 0: the two definitions rank defenses broadly alike. The CI excludes 1 in 7 of 9 cells on the full data (6 of 8 defined cells in the subset), so the rankings are not identical. The preregistration sets no τb threshold; this is descriptive inference. In the subset, gpt-5-mini's EXEC ASR is 0 for all 8 defenses, so τb is undefined.

5.2 Decision rule: flip counts across all analyses (no single headline number)

ATT flips: cells with $\geq$1 flip / flips / pairs tested:

Analysis	Full data (primary; DEVIATION 1 map)	Subset without shopping/github [DEVIATION]
Preregistered rule (episode bootstrap)	4/9 / 6 / 299	3/9 / 4 / 290
Drop-errors (preregistered sensitivity)	4/9 / 6 / 299	3/9 / 4 / 290
Task-clustered paired bootstrap + Holm [EXPLORATORY]	1/9 / 1 / 299	3/9 / 4 / 290
Without progent (LODO) [EXPLORATORY]	1/9 / 1 / 235	1/9 / 1 / 227
Excluding unversioned AgentDojo runs [EXPLORATORY]	4/8 / 6 / 289	3/8 / 4 / 280

Reading the table:

- The preregistered ≥25% threshold is met on the full data (4/9) and in the subset (3/9).
- It is not robust. Task-clustered inference on the full data leaves 1/9 cells (5 of 6 flips disappear), and dropping progent leaves 1/9 in either data set.
- In the subset, dropping any one of drift, piguard_detector or tool_filter gives 2/9, also below 25%, so the threshold there has one cell of margin; on the full data, dropping drift or tool_filter or repeat_user_prompt leaves 3/9.
- Drop-errors is identical to the primary analysis because the analysed cells contain 1 error episode (full) and 0 (subset) of the 92 in the inventory.
- Excluding unversioned runs removes AgentDojo entirely: all 3145 episodes of its only eligible cell come from run files without a benchmark version. AgentDyn flip pairs are unchanged.
- The clustered count is higher in the subset than in the full data because each AgentDyn cell adds 40 shopping/github user tasks, with smaller defense differences, which shrinks pooled estimates, and borderline Holm p-values cross the threshold (Appendix C).

A permutation max-statistic family-wise check [EXPLORATORY; procedure hash-locked before running] flags 12 pairs in 5/9 cells on the full data and 8 pairs in 4/9 cells in the subset. Three full-data flags (two in the subset) involve neither progent nor drift (llama piguard_detector vs tool_filter and tool_filter vs transformers_pi_detector; gpt-5.1 prompt_guard_2_detector vs repeat_user_prompt). Its null is full exchangeability of defenses, so it is less conservative than the preregistered rule and is not a test of opposite-signed non-zero differences; we report it for completeness, not as support.

5.3 Which pairs reverse

- Full data, preregistered rule (6 flips): gpt-4o-2024-08-06 progent vs tool_filter; gpt-4o-mini progent vs repeat_user_prompt; llama-3.3-70b drift vs progent and progent vs tool_filter; qwen3-235b drift vs none and drift vs progent. Progent is in 5 of 6, drift in 3 of 6; every flip involves at least one of them.
- Subset [DEVIATION] (4 flips): gemini-2.5-pro drift vs piguard_detector (ΔEXEC -0.050, ΔATT $+0.171$); gpt-4o-2024-08-06 progent vs tool_filter ($-0.065 \rightarrow +0.387$); llama-3.3-70b drift vs progent ($+0.100 \rightarrow -0.290$) and progent vs tool_filter ($-0.060 \rightarrow +0.360$). The full and subset flip sets are not nested: the gemini-2.5-pro flip occurs only in the subset.
- Task-clustered: the one full-data flip is gpt-4o-2024-08-06 progent vs tool_filter, the largest full-data reversal (ΔEXEC -0.027, ΔATT $+0.219$).

5.4 Validity of the ATTEMPTED scorer on progent and drift [AI LABELS]

The earlier scorer validation (96/113, κ 0.645) covered only AgentDyn gpt-4o episodes under tool_filter, PG2 and repeat_user_prompt, i.e. not the defenses that carry every reversal. We therefore validated ATT on progent and drift episodes:

- *Population:* AgentDyn important_instructions episodes, all suites, in the 4 model cells with a full-data flip (gpt-4o-2024-08-06, gpt-4o-mini, llama-3.3-70b, qwen3-235b); 2240 progent and 2240 drift episodes.
- *Sample:* stratified by defense × scorer label, 20 per stratum, n = 80, seed 20261016.
- *Labelling:* the AI assistant labelled each episode by reading the injection goal and the agent's tool calls, with scorer label, executed flag and stratum hidden; no LLM API was called. **These are AI labels by the same assistant that built the scorer, not human labels.** Rule: attempted = a tool call whose function and purpose pursue the injection goal's effecting action, regardless of whether it was blocked or its arguments were exact.
- *Lock:* labels were hash-locked (sha256 `ca8e4694...5a7dee`, 2026-10-16 14:55:45 Tehran) before the key was opened; the comparison script asserts the hash.

Defense	n	Agreement	Wilson 95%	Cohen κ	Scorer-positive confirmed	Scorer-negative confirmed
progent	40	37/40 (92.5%)	0.801–0.974	0.85	17/20	20/20
drift	40	30/40 (75.0%)	0.598–0.858	0.5	10/20	20/20

All 13 disagreements are scorer false positives: the name match fires on a target-named call that serves the user task (a payment to the user's own payee, a checkout of the user's product, an e-mail to a colleague). Scorer-positive precision is 0.85 (Wilson 0.64–0.948) for progent and 0.50 (0.299–0.701) for drift. Agreement and κ are computed on the stratified sample (equal strata), not population-weighted. **For drift, about half of the scorer's attempts are not attempts at the injection goal, so drift's ATT ASR is inflated by the scorer itself, and the drift reversals (3 of 6 full-data flips) may be scorer artefacts.** For progent the scorer is better but not exact. We did not re-run the flip analysis with corrected labels.

5.5 Interpretation

Every flip under the preregistered and clustered rules involves one of the two execution-time defenses, and the one AgentDojo cell, which has neither, has none. This is **consistent with a mechanical explanation** (an attempt metric gives no credit for blocking at execution time); **we did not test it**, the stage map was written post hoc, and two defenses cannot separate stage from defense identity. The scorer validation adds a second, competing explanation for drift. Stage-level ATT−EXEC gaps are in Appendix B.

Deviations affecting §5. (1) Post-hoc target map for AgentDyn shopping/github, written after first-pass disagreements had been seen. (2) Post-hoc subset excluding shopping/github, reported as a sensitivity analysis only. (3) Unversioned AgentDojo runs were scored as v1; the sensitivity analysis above removes them. (4) The preregistered blind re-label of 53 episodes was not performed. (5) No mixed model was fitted.

6 Short results

6.1 Checker-version drift [EXPLORATORY; negative]

We replayed the logged tool calls of all scorable episodes against current checkers (AgentDojo 089ed468, AgentDyn 5353cf761). Drift is **small in aggregate (0.27%; 198 of 73,009 labels: AgentDojo 39, AgentDyn 159), but up to 7.1 pp in one run set** (AgentDyn gpt-4o-mini repeat_user_prompt, −0.071 [−0.095, −0.048]); 6 of 120 sets have a CI excluding zero, and strict defense order changed in 2 of 15 multi-defense cells, both near-ties. One cause is confirmed by code diff (github injection_task_1 moved from a state- to a trace-based check); 166 changes have no attributable cause. Replay fidelity is limited (OTP feeding; tool outputs not compared with logs), and a first pass that re-executed blocked calls was discarded [DEVIATION; src: docs/DEVIATIONS_2026-10-16.md].

6.2 Judge-vs-checker case study [EXPLORATORY]

On 600 AgentDyn gpt-4o attack episodes (200 per defense, 3 defenses), two LLM judges (gemini-2.5-flash-lite, claude-haiku-5.5; 1,396 calls, $0.3787) agreed with the state checker at κ 0.932 and 0.940 on repeat_user_prompt but 0.454 and 0.632 on tool_filter. A single AI reviewer (one AI reviewer, one model, three defenses; defense identity not hidden) adjudicated the 43 disagreements plus 10 controls and favoured the checker in 20/20 tool_filter and 11/13 PG2 disagreements, but the judge in 3/6 repeat_user_prompt items. **Human check of the AI reviewer.** The owner, blind to the AI reviewer's labels and the key, independently labelled a 16-item subset of the disagreement and control items (`review/owner_subset_ids.json`). Owner and AI reviewer agreed on 14/16 items (87.5%; Wilson 95% CI 64.0–96.5%), Cohen's κ = 0.62. Both disagreements (R15, R26) were items the AI reviewer marked *unclear* and the owner marked

executed (a confirmed password change). With 16 items this is a coarse check and does not replace an independent second rater on the full set. Only 3 of the 16 items were labelled *executed* by the owner, so κ is unstable and should be read as indicative only. The judges' typical error was to count attempts (stalled OTP checkouts, failed forms, text-only mentions) as successes, the same attempted-vs-executed distinction as §5. No claim beyond this one model and three defenses is made.

7 Limitations

- **Almost no human labels.** Every label in this paper (53 disagreement and control items, 60 recall items, 113 earlier scorer-validation items, 80 progent/drift scorer-validation items, 80 ATT-ARG validation items) comes from one AI assistant, which also designed the analyses, the preregistration, the target map and the stage map. The only human check is a blind 16-item owner subset of the judge-review items (14/16 agreement, $\kappa = 0.62$; §6.2, `review/owner_agreement.json`); no other label was checked by a human. Blinding was imperfect in the judge review (defense identity visible) and the labeller of the scorer validation knew how the scorer works.
- **One benchmark in substance.** The ranking result rests on eight AgentDyn cells; AgentDojo contributes one cell without execution-time defenses, and none if unversioned runs are excluded.
- **Two execution-time defenses.** Progent and drift carry every reversal under the preregistered and clustered rules; stage cannot be separated from defense identity.
- **Statistics.** The preregistered bootstrap resamples episodes, not user tasks, and applies no multiplicity correction across 299 (290) pairs; each AgentDyn subset cell has only 20 user-task clusters; the task-clustered analysis is the more appropriate one and gives 1/9 on the full data. The judge bootstraps are not clustered.
- **Scorer validity.** Name-match ATT is tool-identity scoring; on drift its positive labels agree with AI labels only half of the time (§5.4). ATT-ARG is post hoc, excludes 27 task-versions, and its validation uses AI labels on a sample stratified by its own label (§5.0); the human packet is unlabelled.
- **Novelty.** The shopping stubs were identified in AutoDojo's code before us; the exposure census is incomplete (no citation index available).
- **Reference-solution audit.** The 20 defects are confirmed by execution at one commit; user-task reference solutions were probed but not audited task by task; maintainer confirmation is pending.
- **Defective tasks in the analysed data.** The ranking analyses retain dailylife injection_task_2 and injection_task_6, because the preregistration did not provide for removing them.

8 Ethics and disclosure

Who did what. The owner (name withheld for double-blind review) specified the studies, approved each analysis round and its budget caps, and will approve any external communication. An AI assistant (Grok Bot, agent sessions) wrote the analysis code, ran all analyses, wrote the preregistration and sensitivity-procedure texts, produced every label as a single AI labeller, did the literature reads and drafted this manuscript. The assistant is not an author. An independent AI audit of an earlier draft (audit/) drove this revision.

Upstream disclosure. The AgentDyn defects (§4) are drafted as an issue and **not posted**; it will be filed before any preprint, after the owner approves the text [PLACEHOLDER: issue status]. The defects concern measurement only and give an attacker no new capability.

API spend. The only paid calls were the judge pilot: 1,396 calls, $0.3787 from token counts (OpenRouter-reported $0.3746), under a $1.00 cap. Everything else, including all analyses added in this revision, was offline and cost $0.

Dual use. No attacks were run against live systems. All re-analysed data are public releases.

9 Reproducibility

All paths are relative to the `pivot-checks/` directory of the repository. Python 3 with numpy; AgentDyn code from `dyn/` (commit 5353cf7) installed in `venv/`. No network is needed except for the judge pilot.

Result	Script / command	Output	Lock
Inventory and scoring	`rankflip/inventory.py`, `rankflip/score_all.py`	`rankflip/inventory.csv`, `rankflip/episodes.jsonl`	—
TaskSuite.check() per suite	`venv/bin/python c_items/run_check.py`; `c_items/run_check_pertask.py`	`c_items/taskSuite_check.json`, `c_items/taskSuite_check_pertask.{json,txt}`	—
dailylife inj2 root cause	`venv/bin/python c_items/inj2_rootcause.py`	`c_items/inj2_rootcause.txt`	—
User-task probe	`venv/bin/python c_items/usertask_probe.py`	`c_items/usertask_probe.{json,txt}`	—
Reference-solution audit (earlier)	`drift/gt_audit.py`	`drift/gt_audit_agentdyn_v1.json`	—
Ranking flips, full data (primary)	`python3 rankflip/analyze.py`	`rankflip/results.json`	prereg sha256 `97bdb074...`
Subset [DEVIATION]; drop-errors	`python3 review/run.py conf`; `review/run_droperr.py conf|all`	`review/results_conf.json`; `review/results_droperr_{conf,all}.json`	(prereg)
τb table	from the two result JSONs	`bitems/tau_table.md`	—
Task-clustered + Holm	`clustered/clustered.py`, `clustered/diag.py`	`clustered/clustered_results.json`, `clustered/diag_results.json`	`docs/SENSITIVITY_CLUSTERED.sha256`
LODO	`lodo/lodo.py`	`lodo/lodo_results.json`	`docs/SENSITIVITY_LODO.sha256`
Excluding unversioned AgentDojo	`venv/bin/python bitems/run_excl_unversioned.py conf|all`	`bitems/results_exclunv_{conf,all}.json`	—
Permutation family-wise check	`venv/bin/python bitems/perm_fwer.py conf|all`	`bitems/perm_fwer_{conf,all}.json`	`bitems/perm_fwer.py.sha256`
Scorer validation (progent/drift)	`bitems/scorer_validation/sample.py`, then labels, then `compare.py`	`comparison.json`, `precision.txt`	`labels_AI.sha256`
Drift replay	`drift/replay_current.py`, `drift/analyze_drift.py`	`drift/DRIFT_RESULTS.md`	—
Judge pilot (paid)	judge_pilot scripts	`judge_pilot/RESULTS.md`	—
Number check	`python3 review/verify_numbers.py <manuscript> <new output file>`	`review/verify_numbers_out_round10.txt`	—
ATT-ARG scoring and analyses	`venv/bin/python attarg/build_table.py`, `attarg/score_attarg.py`, `attarg/analyses.py`	`attarg/summary.txt`, `attarg/out/`	`attarg/PROCEDURE_ATTARG.sha256`
ATT-ARG validation	`attarg/validation/sample.py`, labels, `compare.py`	`attarg/validation/comparison.json`	`labels_AI.sha256`

Result	Script / command	Output	Lock
Extended audit	`venv/bin/python ext/audit_suites.py <src> <out>`; `ext/injectable_probe.py`	`ext/EXT_AUDIT_RESULTS.md`	`ext/PROCEDURE_EXT_AUDIT.sha256`
Exposure census	manual full reads	`exposure/EXPOSURE_TABLE.md`	`exposure/PROCEDURE_EXPOSURE.sha256`
Human packet	`venv/bin/python human_packet/make_packet.py`	`human_packet/`	`human_packet/PROCEDURE_HUMAN_PACKET.sha256`, `KEY_SEPARATE.sha256`

The deviations log is `docs/DEVIATIONS_2026-10-16.md` (#1–#11).

10 Conclusion

Auditing artifacts we did not produce supports two observations about AgentDyn, not general prescriptions.

- **Main:** (shopping part first fixed in AutoDojo's code) 20 of 28 AgentDyn-specific injection tasks ship a reference solution that fails its own security check (18 copy-paste banking stubs, one wrong-argument call, one missing OTP step), confirmed by executing the check logic. The self-check and reference-keyed metrics are broken; released state-based ASRs are not.
- **Secondary:** with the argument-level scorer ATT-ARG, attempt- and execution-based scoring rank defenses broadly alike (τb 0.642–1.0), and the preregistered flip rule finds reversals in 3/9 cells (2/9 in a post-hoc subset), all involving progent and all on AgentDyn; 1/9 under task-clustered inference and 0/9 without progent. The name-match drift reversals do not survive argument-level scoring. This is an empirical instance of the inversion Pathade et al. analyse, on one benchmark.
- **Short:** checker drift is small in aggregate (0.27%) but up to 7.1 pp in one set; in a one-model, three-defense case study, LLM judges counted blocked attempts as successes.

All labels come from one AI assistant except a blind 16-item owner check (14/16 agreement, $\kappa = 0.62$); maintainer confirmation and independent reproduction are pending.

Declarations

Competing interests. [PLACEHOLDER: owner to confirm] The authors declare no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. [PLACEHOLDER: owner to confirm] This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and code availability. All analysed runs are public releases of AgentDojo and AgentDyn. Analysis scripts, manifests with SHA-256 hashes, preregistration and lock files, the deviations log and the number-verification script are in the release archive [PLACEHOLDER: update release.zip with the c_items/ and bitems/ scripts; repository DOI after deposit].

CRediT author statement. [PLACEHOLDER: author name] — Conceptualization, Methodology, Supervision, Project administration, Writing – review and editing. [PLACEHOLDER: owner decides whether to add "Validation"; the owner blind-labelled a 16-item subset only.] The AI assistant is not an author.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work the author used an AI agent (Grok Bot) to write and run the analysis code; write the preregistration and sensitivity-procedure texts; produce all labels except a blind 16-item owner check as a single AI labeller (53 disagreement and control items, 60 recall items, 113 + 80

+ 80 scorer-validation items); read and summarise related literature; and draft the manuscript. The LLM judges in §6.2 are objects of study, not writing aids. [PLACEHOLDER: the standard sentence that the author reviewed and edited the content and takes full responsibility may be added only after the author's own verification step is completed; it has not been done yet; the owner has blind-labelled a 16-item subset (§6.2) but has not otherwise verified the results.]

References

Preprints are marked as such. DOIs are to be added at formatting.

- Debenedetti E, et al. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. arXiv preprint 2406.13352; 2024.
- Zhan Q, et al. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents. arXiv preprint 2403.02691; 2024.
- Debenedetti E, et al. Defeating Prompt Injections by Design (CaMeL). arXiv preprint 2503.18813; 2025.
- Shi T, et al. Progent: Securing AI Agents with Privilege Control. arXiv preprint 2504.11703; 2025.
- Shaw A. Silent Failures in Agentic Security Evaluation: A Validated Harness for Tool-Call Mediation Under Indirect Prompt Injection. arXiv preprint 2609.32691; 2026.
- Pathade C, Pawar P, Patil S. Attack Success Rate Is Not a Number: On Measurement Validity in Agentic AI Security Evaluation. arXiv preprint 2609.25173; 2026.
- Yin Y. The Same Zero: Why Identical ASR Can Imply Different Guarantees in LLM-Agent Security. arXiv preprint 2610.04504; 2026.
- Liu Z. Passing the Test You Trained On: Re-evaluating Prompt-Injection Detectors for LLM Agents. arXiv preprint 2610.03448; 2026.
- Hardy M, Azam R, Reuel A, Kochenderfer M, Koyejo S. Agent Evaluation Reliability: More Tasks Won't (Always) Fix an Agent Leaderboard. arXiv preprint 2610.00651; 2026.
- Li H, Wen R, Shi S, Zhang N, Vorobeychik Y, Xiao C. AgentDyn: Are Your Agent Security Defenses Deployable in Real-World Dynamic Environments? arXiv preprint 2602.03117v3; 2026. Code: https://github.com/SaFo-Lab/AgentDyn (github.com/leolee99/AgentDyn redirects there).
- Bellibatlu RR, Wang Z, Zhang W. Do Agent Benchmarks Do What They Say? An Executable-Contract Audit of Tool-Using Agent Environments. arXiv preprint 2609.37315; 2026.
- BenchGuard: Who Guards the Benchmarks? Automated Auditing of LLM Agent Benchmarks. arXiv preprint 2604.24955; 2026 [author list to verify].
- Vaghasiya J, Bhat V, Mohsin MA, Aali A. Benchmarking the Benchmarks: A Validity Audit of Tool-Calling Evaluation. arXiv preprint 2607.02577; 2026.
- Abrich. Finding Blind Spots in AppWorld and WorkArena Task Verifiers. arXiv preprint 2610.09142; 2026 [full author list to verify].
- Luo W, Luu K, et al. Maintaining Benchmarks Against Increasingly Capable Agents: Detection and Remediation of Unearned Passes. arXiv preprint 2609.34262; 2026.
- Zhu Y, et al. Establishing Best Practices for Building Rigorous Agentic Benchmarks. arXiv preprint 2507.02825v5; 2025.
- AutoDojo. arXiv preprint 2606.15057; 2026 [author list to verify]. Code: https://github.com/xhOwenMa/AutoDojo (commit aa458794).
- Agent-Sentry: Bounding LLM Agents via Execution Provenance. arXiv preprint 2603.22868; 2026 [authors to verify].
- Aligning Provenance with Authorization: A Dual-Graph Defense for LLM Agents (AuthGraph). arXiv preprint 2605.26497; 2026 [authors to verify].
- ActGov: Governing LLM Agent Actions via Policy-Constrained Validation. arXiv preprint 2609.24446v2; 2026 [authors to verify].
- Supplementary S1: the own-harness study that motivated this audit (v1 manuscript and reference list).

Appendix A: All numbers and sources

Number	Meaning	Source
125 / 74,528; 50 / 75	run sets / injection episodes; AgentDojo / AgentDyn sets	rankflip/inventory.csv; drift/DRIFT_RESULTS.md; docs/DEVIATIONS_2026-10-16.md
73,009 (31,604 + 41,405)	non-error, non-DoS replayed episodes	drift/DRIFT_RESULTS.md
20 of 28; 9 + 9 + 2	AgentDyn-specific injection tasks failing their own check, confirmed by execution	c_items/C_RESULTS.md; drift/gt_audit_agentdyn_v1.json
6/6, 7/7, 9/9, 5/5	AgentDojo-suite injection tasks passing	c_items/C_RESULTS.md
1810.0; 500	inj2 amount; OTP threshold	c_items/inj2_rootcause.txt; c_items/C_RESULTS.md
162/1460	inj2 episodes with security True in released runs	c_items/C_RESULTS.md
20/20, 13/20, 9/20	user tasks failing utility after reference (shopping, github, dailylife)	c_items/C_RESULTS.md
0.514–0.889; 0.8	τb full data, AgentDyn range; AgentDojo	bitems/B_RESULTS.md; rankflip/results.json
7 of 9; 6 of 8	cells whose τb CI excludes 1 (full; subset)	bitems/B_RESULTS.md
4/9 cells, 6 flips / 299 pairs; 3/9, 4 flips / 290 pairs	preregistered rule: full; subset [DEVIATION]	rankflip/results.json; review/results_conf.json
1/9 cells, 1 flip / 299 pairs; 3/9, 4 flips / 290 pairs	task-clustered + Holm: full; subset	clustered/clustered_results.json
1/9 cells, 1 flip / 235 pairs; 1/9, 1 flip / 227 pairs	without progent: full; subset	lodo/lodo_results.json
4/8 cells, 6 flips / 289 pairs; 3/8, 4 flips / 280 pairs; 3145	excluding unversioned AgentDojo runs; episodes removed	bitems/B_RESULTS.md
12 in 5/9; 8 in 4/9; 3; 2	permutation flags full; subset; flags without progent/drift	bitems/B_RESULTS.md
$-0.027 \rightarrow +0.219$	largest full-data reversal, gpt-4o-2024-08-06 progent vs tool_filter	review/RANKING_FLIP_RESULTS.md §3
37/40, 92.5%, 0.801–0.974, κ 0.85	scorer validation, progent	bitems/B_RESULTS.md
30/40, 75.0%, 0.598–0.858, κ 0.5	scorer validation, drift	bitems/B_RESULTS.md
0.85 (0.64–0.948); 0.50 (0.299–0.701)	scorer-positive precision progent; drift	bitems/scorer_validation/precision.txt
2240; 20261016	population per defense; sample seed	bitems/B_RESULTS.md
96/113, κ 0.645	earlier scorer validation (tool_filter/PG2/repeat)	review/RANKING_FLIP_RESULTS.md §2
198 / 73,009 (0.27%); −0.071; 6/120; 2 of 15; 166	checker drift	drift/DRIFT_RESULTS.md
1,396; $0.3787 ($0.3746); 600	judge pilot calls, spend, episodes	judge_pilot/RESULTS.md
κ 0.454 / 0.632 / 0.932 / 0.940	judge–checker agreement	judge_pilot/RESULTS.md
43 + 10; 20/20; 11/13; 3/6	single-AI adjudication	review/REVIEW_RESULTS.md
0.081 / 0.040 / 0.154	stage gaps (Appendix B)	stage/STAGE_RESULTS.md
97bdb074…61e63c1	prereg hash	review/PREREG_RANKFLIP.sha256

Number	Meaning	Source
3/9, 5 / 299; 2/9, 4 / 290	ATT-ARG preregistered rule: full; subset	attarg/summary.txt
1/9, 1; 2/9, 3; 0/9	ATT-ARG clustered full; clustered subset; without progent	attarg/summary.txt
0.642–1.0	ATT-ARG τb range, full data	attarg/summary.txt
27 of 154; 9,080; 1,460	excluded task-versions; excluded AgentDojo / AgentDyn episodes	attarg/counts.txt
78/80 (κ 0.95); 40/40; 76/80 (κ 0.9)	ATT-ARG validation; precision; name-match on same sample	attarg/validation/comparison.json
8; 9; 1; 1; 9; 3	extended-audit counts (AgentDojo stub; AgentDyn stub, wrong args, missing step, tool absent; AutoDojo other)	ext/EXT_AUDIT_RESULTS.md
5; 2; 1; 2	AgentDyn papers found; exposed; not exposed; unknown	exposure/EXPOSURE_TABLE.md
0 / 0 / 0	verify_numbers.py mismatches (round 10, new output file; with ATT-ARG/audit hard checks)	review/verify_numbers_out_round10.txt

Appendix B: Stage-level ATT−EXEC gaps [EXPLORATORY]

Mean ATT−EXEC gaps (important_instructions, error episodes dropped, 9 cells, stage map written after the flips were seen): input 0.081, planning 0.040, execution-time 0.154 (two defenses only). No p-values are reported; with two execution-time defenses the contrast cannot be generalised beyond progent and drift, and the ATT values for drift are affected by the scorer problem in §5.4.

Appendix C: Why clustered inference gives more flips in the subset than in the full data

The full data is not a superset test. Each AgentDyn cell adds 40 shopping/github user tasks (60 vs 20 clusters), whose defense differences are smaller, so pooled estimates shrink: e.g. gemini-2.5-pro drift vs piguard_detector ΔEXEC goes from -0.050 to -0.009, and llama drift vs progent ΔATT from -0.29 to -0.093. Borderline cases cross the Holm threshold in both directions; gpt-4o-2024-08-06 progent vs tool_filter has ΔEXEC p = 0.0062 in the subset (not significant at m = 36) and 0.0042 in the full data (significant).

Appendix D: Open issues

1. Human validation covers only the 16-item owner subset (14/16, $\kappa = 0.62$); no second human rater on the full label sets.
2. AgentDyn maintainer issue awaiting owner approval (tracker checked 2026-10-16; no prior report of these defects).
3. Flip analysis not re-run with corrected ATT labels for drift/progent.
4. Several new reference author lists to verify; Zenodo folder built (`pivot-checks/release_zenodo/`), license placeholder.
5. User-task reference solutions: failures observed, not root-caused per task; `check()` "not injectable" behaviour root-caused (str-only filter).
6. Human packet (40 episodes) awaiting a labeller; exposure census incomplete.

Erratum (2026-10-17)

- OLD (exposure table, Agent-Sentry row): "exposed".
- NEW: "plausibly exposed (inference; paper does not state use of ground_truth())".
- Evidence: `exposure/EXPOSURE_TABLE.md` (erratum entry, OLD/NEW); number check of this text: `review/verify_numbers_out_owner_20261019.txt`.